
Artificial Intelligence

Tut/Lab week 3

COSC2129/3118

Outline

- Warm up
- Highlights:
 - Problem solving by search
 - Search & games
 - Adversarial search
- Hands-on exercise
- Assignment 1

Material

- Chapter 6, Book: Artificial Intelligence: A Modern Approach Global edition
- Book - TOC of Artificial Intelligence: A Modern Approach, 4th Global ed.
- AIMA: Pseudo code
- Good practices Python

Questions may be asked in a job interview

- Explain the concept of adversarial search in AI. How does it differ from traditional search algorithms?
- Explain the role of Alpha-Beta pruning in improving the efficiency of adversarial search. How does it reduce the time complexity, and in which scenarios might it not be effective?
- In adversarial search, what is the difference between a deterministic and a stochastic game, and how does this affect the choice of search algorithms?
- In the Pac-Man game, how would you apply adversarial search to help Pac-Man navigate through the maze, avoiding ghosts while collecting pellets?
What adversarial search algorithms would you consider, and what factors would you take into account when making decisions?
- Applications of adversarial search

Breaking news?

- Grok 4 Is Now #1 in AI —This Changes Everything
- A golden moment!
- EU AI Act's Code of Practice Finally Published After Months of Delays | Regulatory Milestone
The EU published its Code of Practice for General-Purpose AI last week

Grok 4

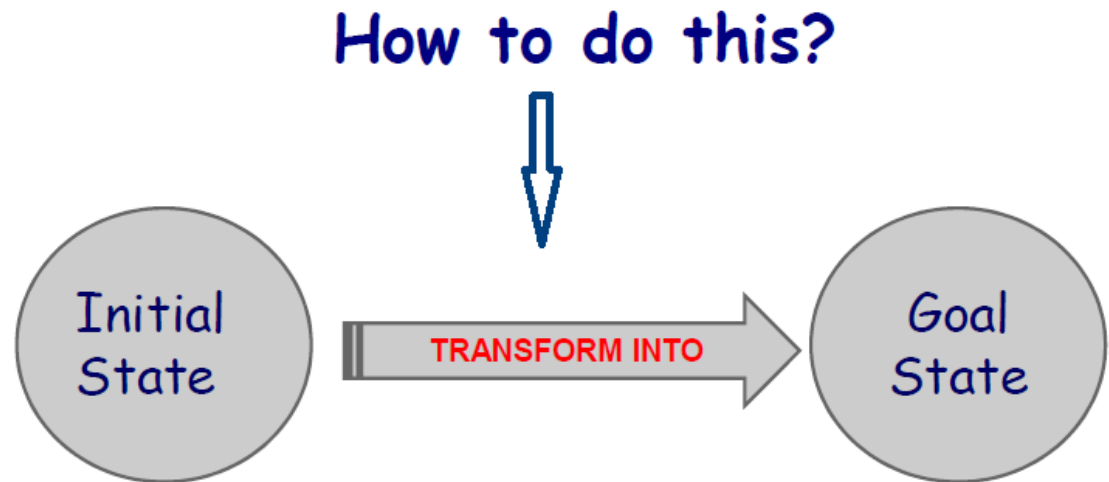
- Can it do reasoning?
- Can it do planning?

Breaking news (2024)

- More than 25% of new code at Google is now generated by AI (Google CEO Sundar Pichai)
- Is Game Over for Copyright Claims?
 - OpenAI scores key legal victory as judge throws out copyright case brought by news websites
 - ChatGPT creates synthesized responses from its training rather than copying content directly
 - factual information in articles isn't copyrighted
 - the likelihood of ChatGPT reproducing exact copies of articles is minimal
- ...
- The Future of AI (8th FII conference)

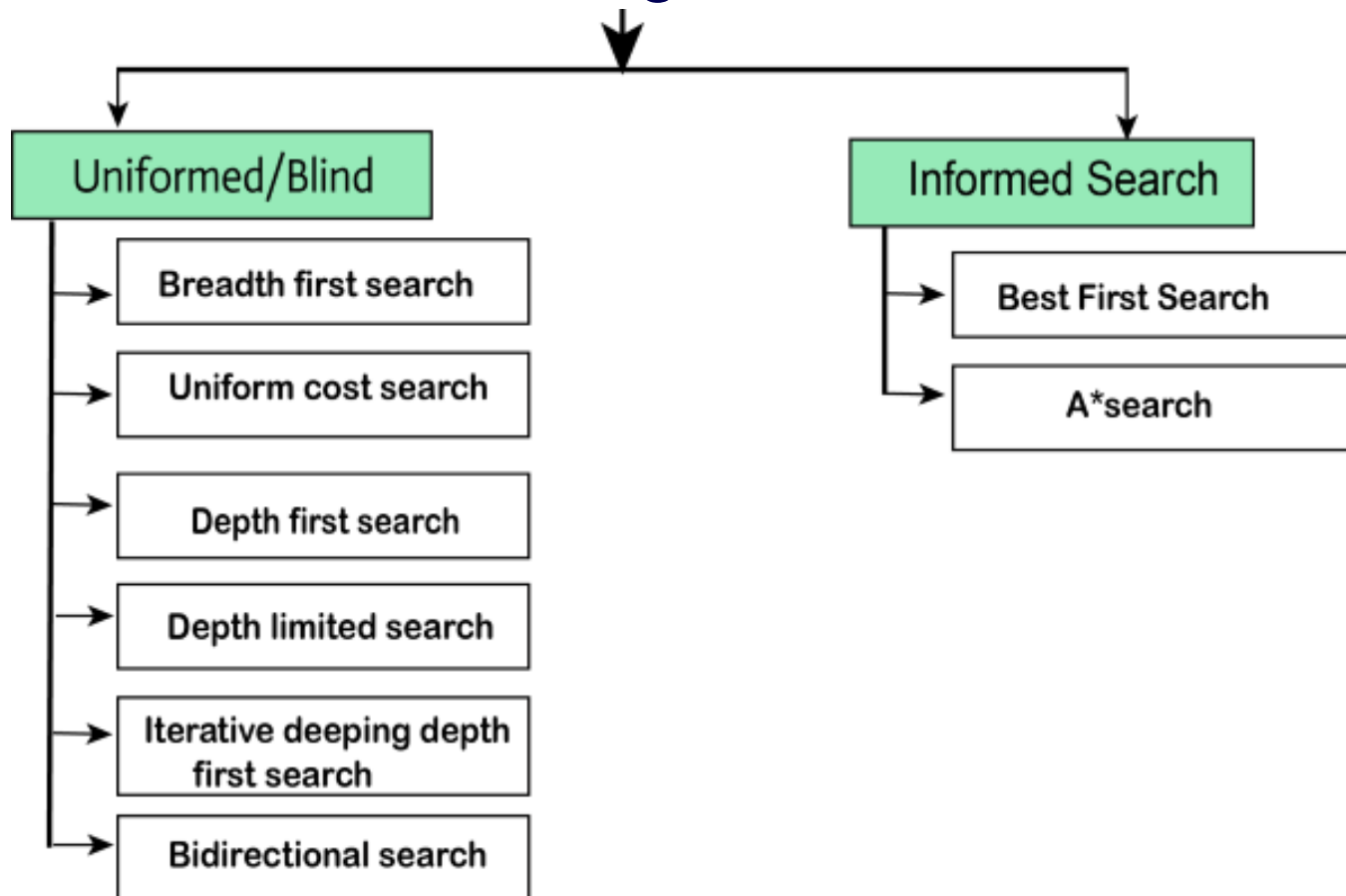
Search problems

- A search problem is defined by:
 - Possible states (search space)
 - Initial state
 - Actions
 - Transition mode
 - Goal test
 - Path cost



Blind search algorithms

Search Algorithms



Performance of search strategies

- **Completeness:** Is it guaranteed to find a solution if one exists?
- **Optimality:** Will it find an optimal solution?
- **Time complexity:** How many nodes get expanded?
- **Space complexity:** How much memory is needed?
- **Time and space complexity is calculated based on:**
 - b : Branching factor
 - d : Depth of the shallowest goal node
 - m : Maximum length of any path in the state space.

Search & games

- Search & games
- Adversarial search
- Minimax
- Alpha-beta pruning
- H-minimax
- Expectimax.

Games: State-of-the-art

Checkers:

- 1950: First computer player
- 1959: Samuel's self-taught program
- 1995: Chinook: first computer world champion*
- 2007: Checkers solved!

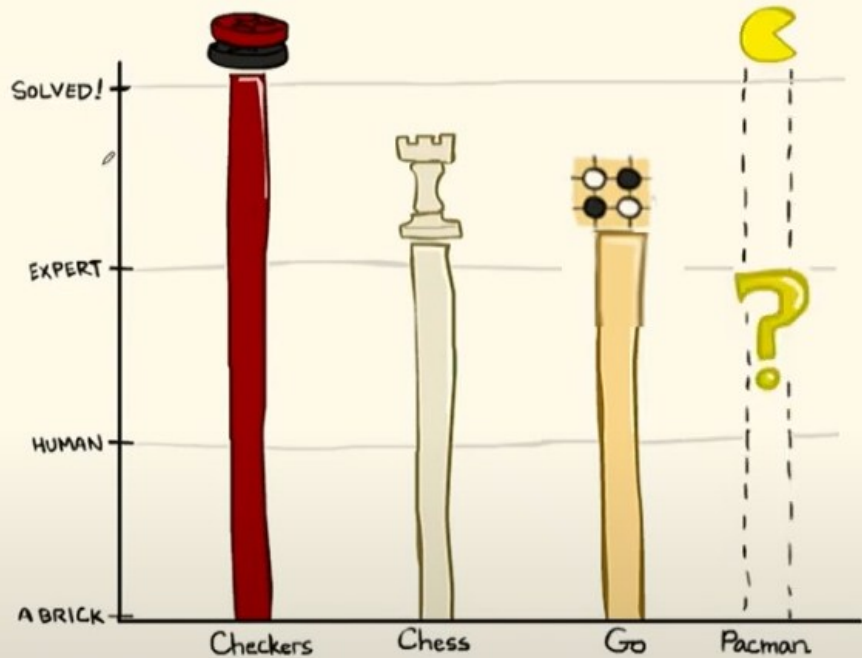
Chess:

- 1945-1960: Zuse, Wiener, Shannon, Turing, Newell & Simon, McCarthy.
- 1960-1996: gradual improvements
- 1997: Deep Blue defeats human champion Garry Kasparov
- 2024: Stockfish rating 3631 (vs 2847 for Magnus Carlsen)

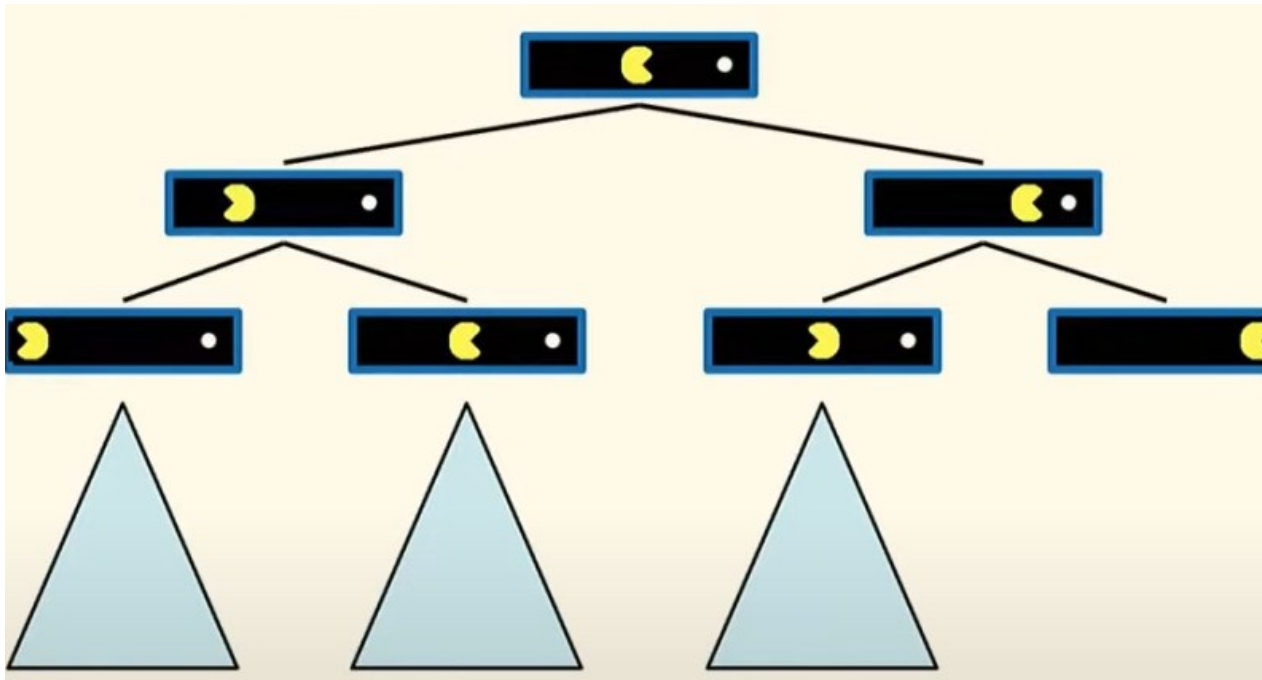
Go:

- 1968: Zobrist's program plays legal Go, barely (b>300!)
- 1968-2005: various ad hoc approaches tried, novice level
- 2005-2014: Monte Carlo tree search -> strong amateur
- 2016-2017: AlphaGo defeats human world champions
- 2022: Human exploits NN weakness to defeat top Go programs

Pacman

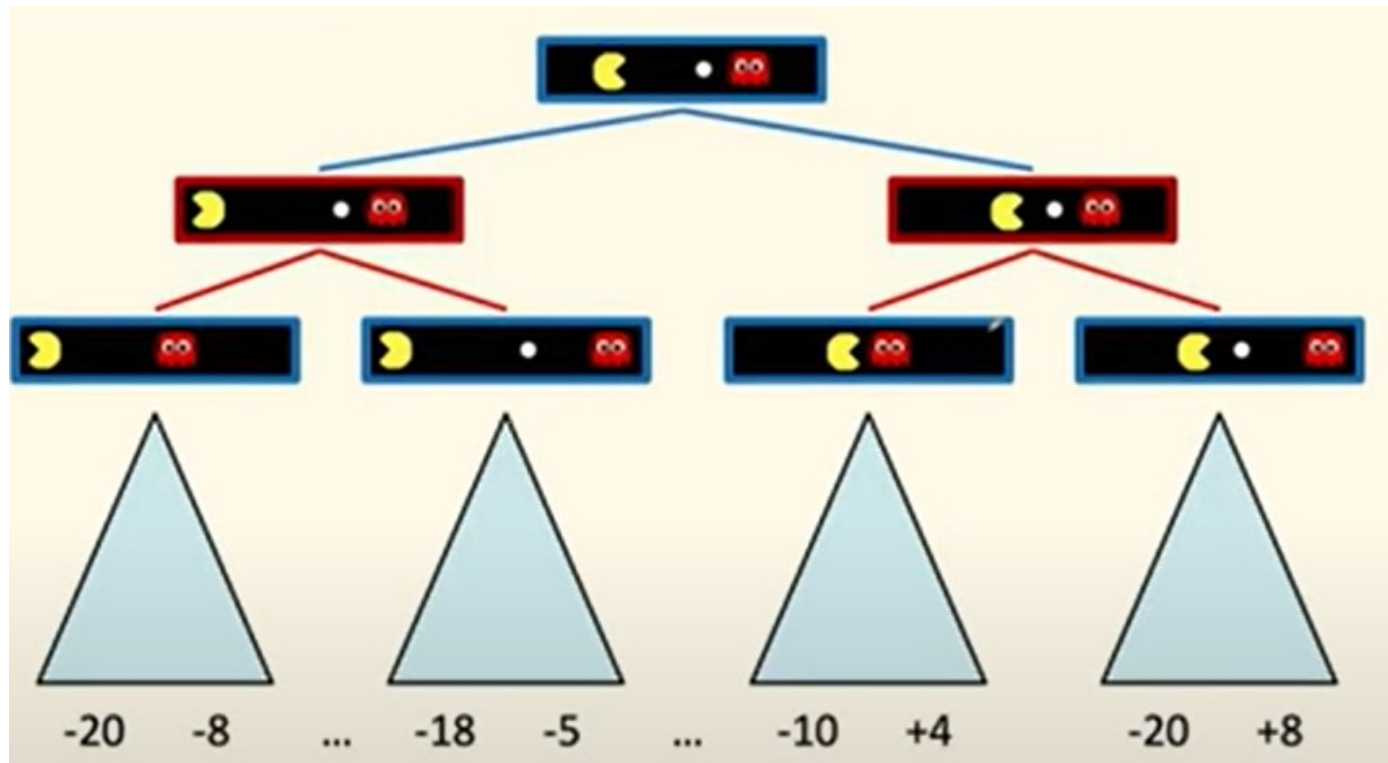


Single agent

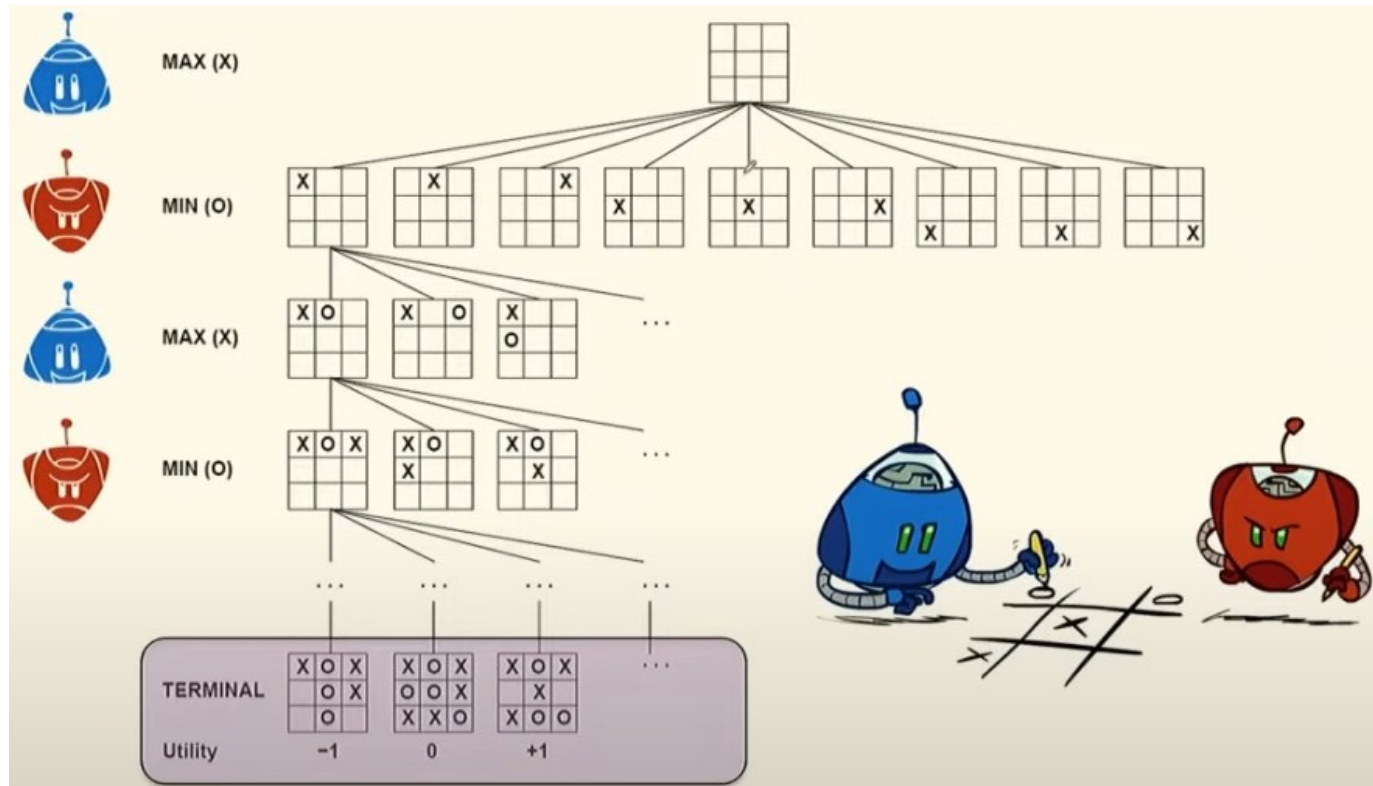


Cre: CS188

Adversarial game



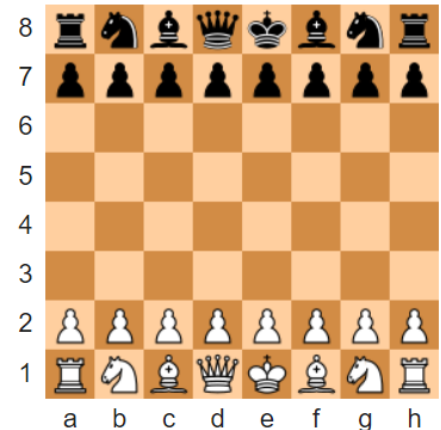
Tic tac toe game



Games and search

Games

- **States:** S with initial state s_0
- **Player(s):** P (who moves in state s)
- **Action(s):** A (legal moves a in S)
- **Result(s,a):** transition model ($S \times A \rightarrow S$)
- **Terminal-test(s):** true if the game is over
- **Utility(s):** utility for s if s is terminal ($S \times P \rightarrow R$)
- **Solution** for a player is a **policy**: $S \rightarrow A$



wiki/Chess

Minimax search

Two players: MAX and MIN

Initial state is a MAX node

Children of MAX state are MIN states

Children of MIN state are MAX states

Calculate Minimax(s)

- $Utility(s)$ if $Terminal-test(s)$
- $Max_a Minimax(Result(s,a))$ if $Player(s) = MAX$
- $Min_a Minimax(Result(s,a))$ if $Player(s) = MIN$
- Calculate in depth-first manner

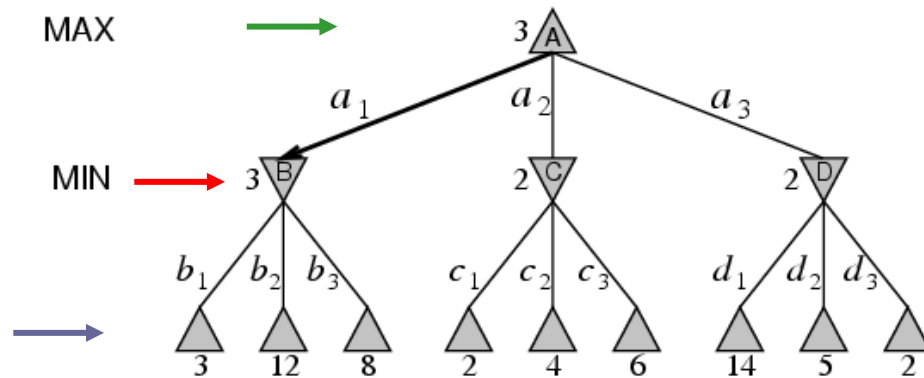
Minimax search

- Leaf node: $\text{Minimax}(s) = \text{Utility}(s)$
- MAX node: $\text{Minimax}(s)$ is **maximum** of children
- MIN node: $\text{Minimax}(s)$ is **minimum** of children

"I choose best move for me, assuming my opponent chooses the best move for him/her".

Minimax search example

Assume, for example, a game that only has two moves from start to finish:



- ▲ is a "MAX node". It is MAX's turn to move (to find the max value).
- ▼ is a "MIN node". It is MIN's turn to move (to find the min value).

Pseudo code

AIMA book, Chapter 6

function MINIMAX-SEARCH(*game*, *state*) **returns** an action
 player $\leftarrow$ game.TO-MOVE(*state*)
 value, move $\leftarrow$ MAX-VALUE(*game*, *state*)
return move

function MAX-VALUE(*game*, *state*) **returns** a (utility, move) pair
 if game.IS-TERMINAL(*state*) **then return** game.UTILITY(*state*, player), null
 v, move $\leftarrow -\infty$
for each a in game.ACTIONS(*state*) **do**
 v2, a2 $\leftarrow$ MIN-VALUE(*game*, game.RESULT(*state*, a))
 if v2 > v **then**
 v, move $\leftarrow$ v2, a
return v, move

function MIN-VALUE(*game*, *state*) **returns** a (utility, move) pair
 if game.IS-TERMINAL(*state*) **then return** game.UTILITY(*state*, player), null
 v, move $\leftarrow +\infty$
for each a in game.ACTIONS(*state*) **do**
 v2, a2 $\leftarrow$ MAX-VALUE(*game*, game.RESULT(*state*, a))
 if v2 < v **then**
 v, move $\leftarrow$ v2, a
return v, move

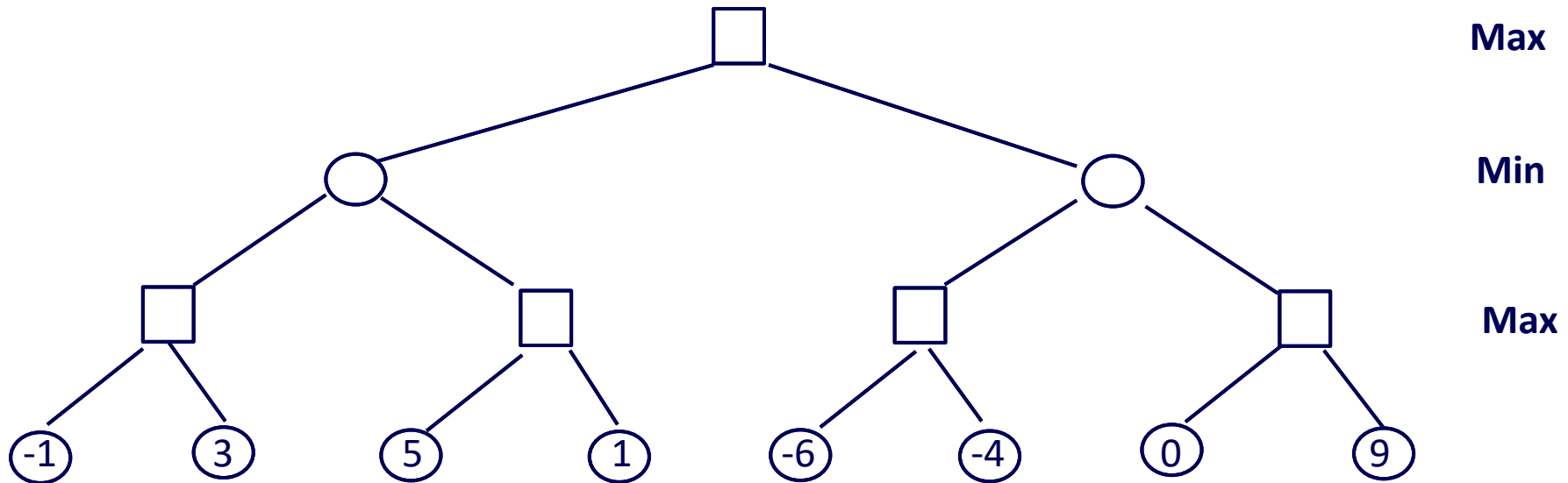
AIMA3e

function MINIMAX-DECISION(*state*) **returns** an action
return arg max_{a ∈ ACTIONS(s)} MIN-VALUE(RESULT(*state*, a))

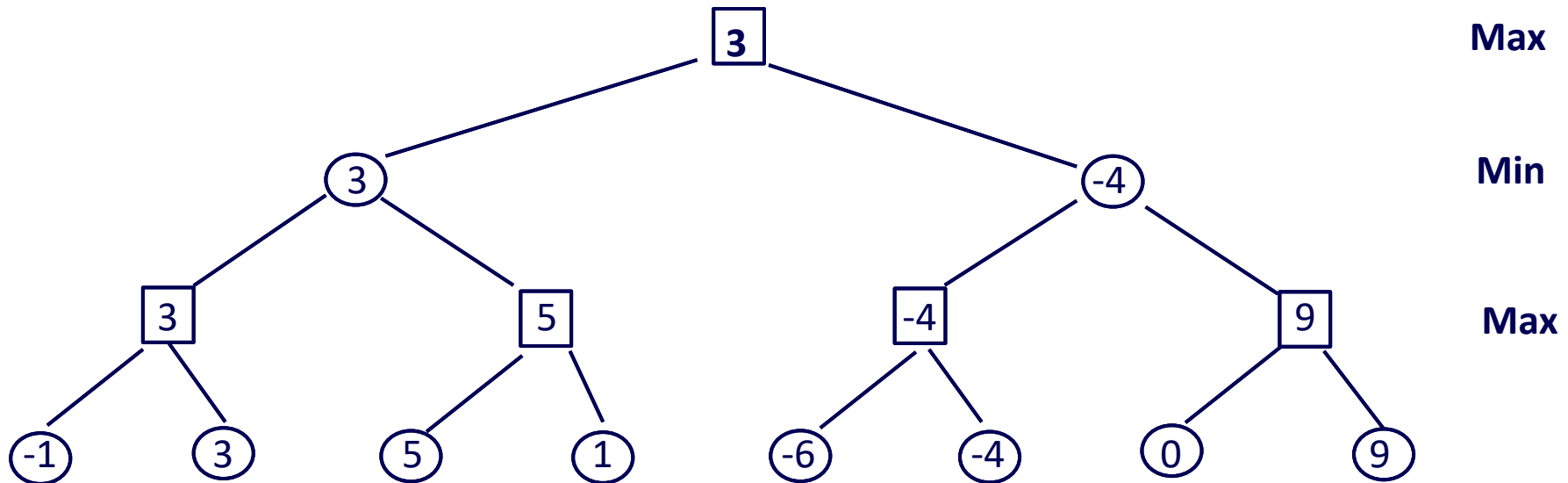
function MAX-VALUE(*state*) **returns** a utility value
 if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
 v $\leftarrow -\infty$
for each a in ACTIONS(*state*) **do**
 v $\leftarrow$ MAX(v, MIN-VALUE(RESULT(*state*, a)))
return v

function MIN-VALUE(*state*) **returns** a utility value
 if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
 v $\leftarrow \infty$
for each a in ACTIONS(*state*) **do**
 v $\leftarrow$ MIN(v, MAX-VALUE(RESULT(*state*, a)))
return v

Minimax - example

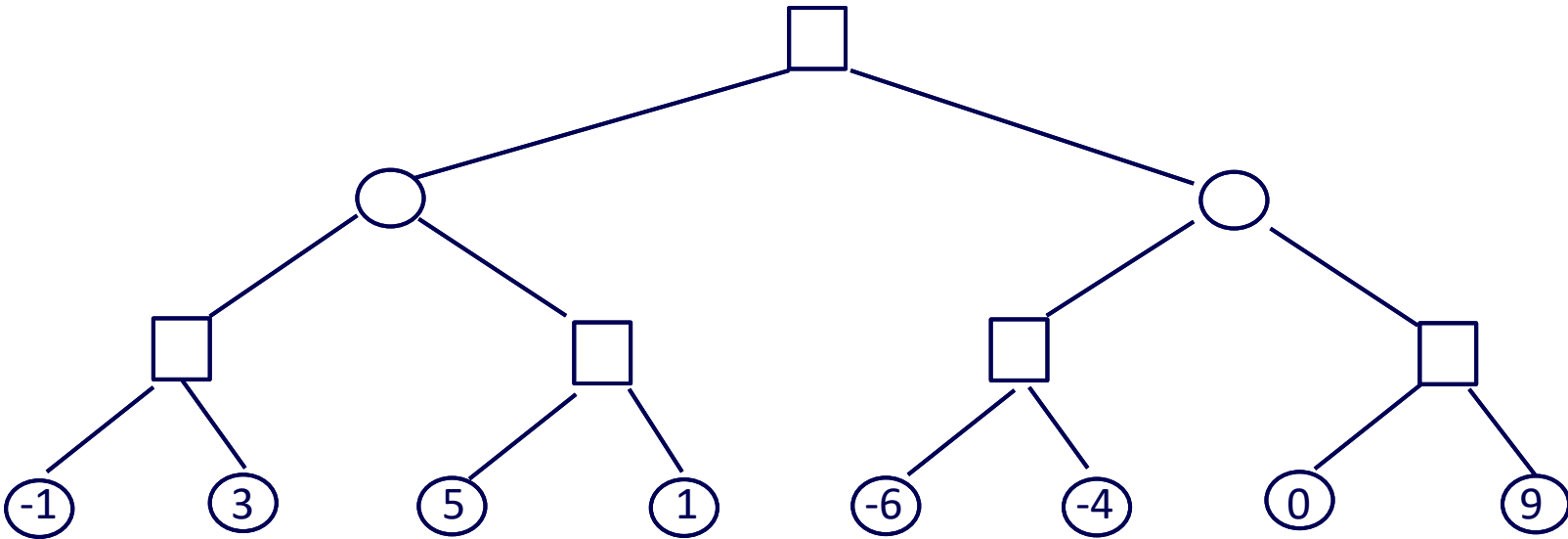


Minimax - example

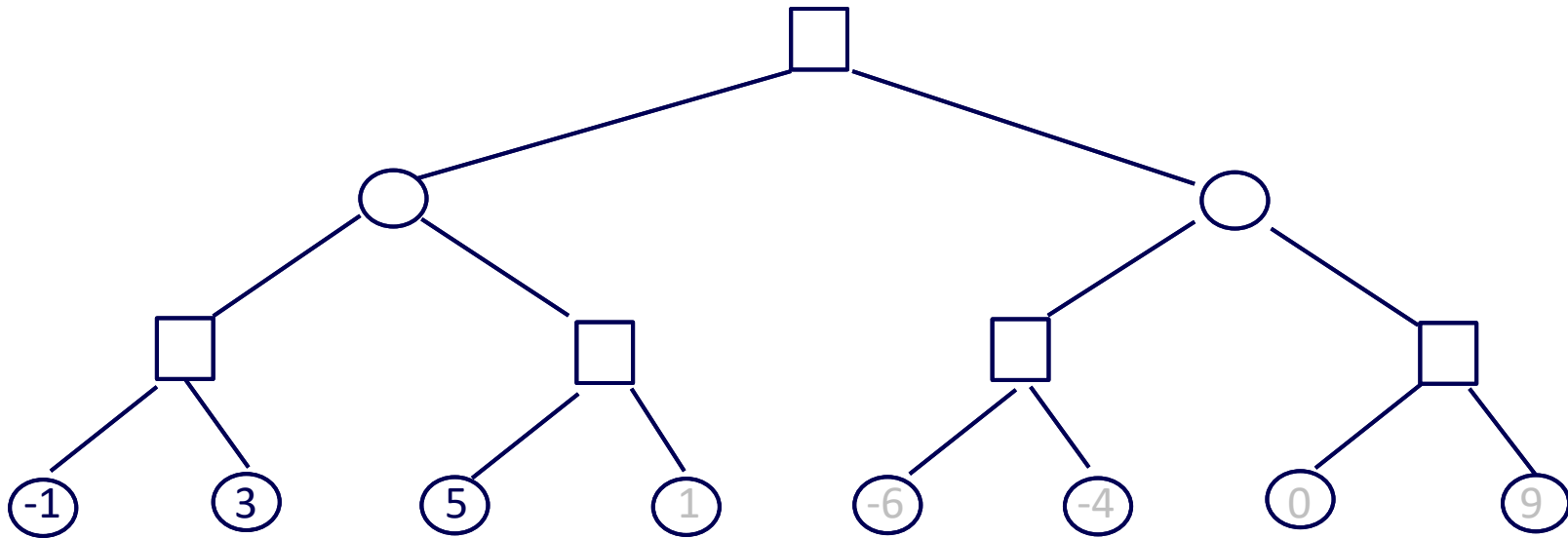


Minimax: Alpha-Beta pruning

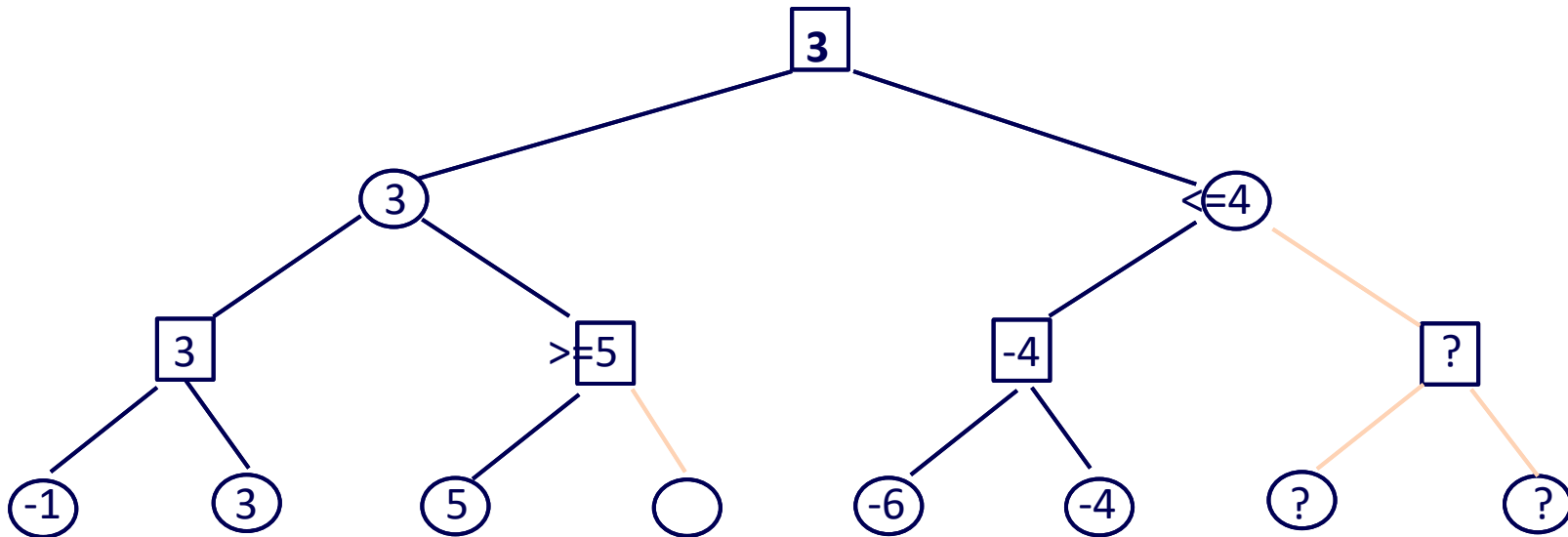
- Which nodes will not be explored?



Alpha-Beta pruning



Alpha-Beta pruning



Alpha-beta pruning

Same search as for minimax except:

MIN node: stop if current value $\leq$ alpha of parent
(this node can't be higher than alpha)

MAX node: stop if current value $\geq$ beta of parent
(this node can't be lower than beta)

This means that the whole subtree for a pruned node is not expanded.

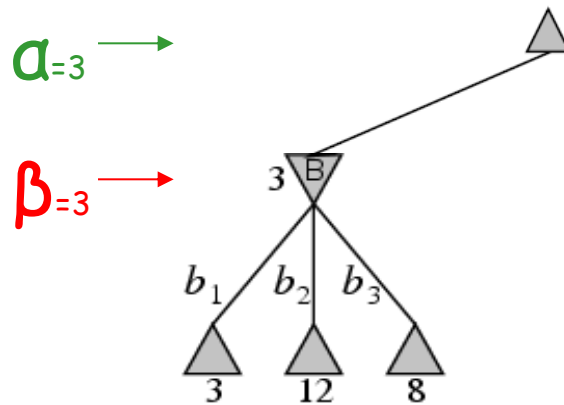
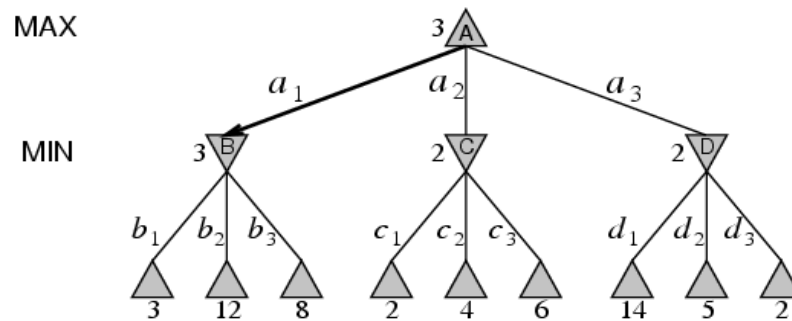
Pseudo code

function ALPHA-BETA-SEARCH(*game*, *state*) **returns** an action
 $\text{player} \leftarrow \text{game.TO-MOVE}(\text{state})$
 $\text{value}, \text{move} \leftarrow \text{MAX-VALUE}(\text{game}, \text{state}, -\infty, +\infty)$
 return *move*

function MAX-VALUE(*game*, *state*, α , β) **returns** a (*utility*, *move*) pair
 if *game.IS-TERMINAL*(*state*) **then return** *game.UTILITY*(*state*, *player*), *null*
 $v \leftarrow -\infty$
 for each *a* **in** *game.ACTIONS*(*state*) **do**
 $v2, a2 \leftarrow \text{MIN-VALUE}(\text{game}, \text{game.RESULT}(\text{state}, a), \alpha, \beta)$
 if $v2 > v$ **then**
 $v, \text{move} \leftarrow v2, a$
 $\alpha \leftarrow \text{MAX}(\alpha, v)$
 if $v \geq \beta$ **then return** *v*, *move*
 return *v*, *move*

function MIN-VALUE(*game*, *state*, α , β) **returns** a (*utility*, *move*) pair
 if *game.IS-TERMINAL*(*state*) **then return** *game.UTILITY*(*state*, *player*), *null*
 $v \leftarrow +\infty$
 for each *a* **in** *game.ACTIONS*(*state*) **do**
 $v2, a2 \leftarrow \text{MAX-VALUE}(\text{game}, \text{game.RESULT}(\text{state}, a), \alpha, \beta)$
 if $v2 < v$ **then**
 $v, \text{move} \leftarrow v2, a$
 $\beta \leftarrow \text{MIN}(\beta, v)$
 if $v \leq \alpha$ **then return** *v*, *move*
 return *v*, *move*

Alpha-beta pruning



- Expand the first branch.
- Get evaluation values for each leaf node.
- So the value for B (β) is 3.
- So the root value (α) is currently 3.

Games and search

- Set up game as a kind of search tree
- Use Minimax method to choose moves
- Use alpha-beta pruning to cut down alternatives
- Some implementation tricks such as transposition tables as hash tables

Games and search

- Games involve search, but ...
- Search space can be huge (chess 10^{40} states (!))
- Not all moves may be chosen by opponent
- Time limits may limit search
- Exhaustive search is not how humans do it!

Games and search

- Assume optimal opponent
- Prune search space
- Use heuristic approximations
- Deal with imperfect information and chance

Evaluation Function

Limit depth and estimate utility via a
heuristic evaluation function (educated guess)

Calculate **H-Minimax(s)**

- **Eval(s)** if **Cutoff-test(s,d)**
- Max_a **H-Minimax(Result(s,a),d+1)** if **Player(s) = MAX**
- Min_a **H-Minimax(Result(s,a),d+1)** if **Player(s) = MIN**

So what is a good evaluation function?

- Efficient
- Match utility on terminal states
- Correlate with winning chances on non-terminal states
- Will have some level of uncertainty (as it is an **estimate**)

Evaluation Function

Often a **weighted sum** of factors

$$\text{Eval}(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

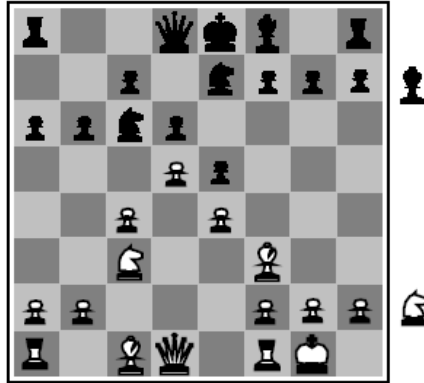
Assumes factors are **independent**

Common chess function:

$$9 * \text{\#Queens} + 4 * \text{\#Bishops} + 3 * \text{\#Knights} + \text{\#pawns}$$

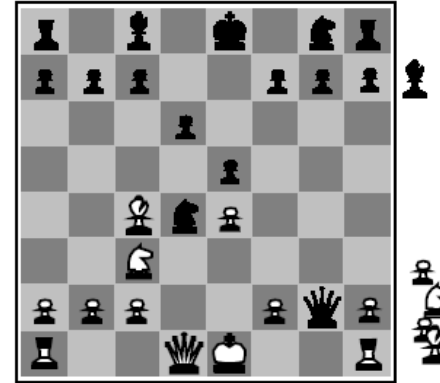
Says nothing about the position of chess pieces.

Evaluation Function



Black to move

White slightly better



White to move

Black winning

For chess the weights and factors could be:

$$w_1 f_1(s) = 9 * (\text{number of white Queen} - \text{number of black queen})$$

$$w_2 f_2(s) = 4 * (\text{number of white Bishop} - \text{number of black Bishop})$$

$$w_3 f_3(s) = 3 * (\text{number of white knight} - \text{number of black knight})$$

$$w_4 f_4(s) = 1 * (\text{number of white pawn} - \text{number of black pawn})$$

... so on and so forth...

By this simple function we can estimate which player is closer to winning.

Activity 2: Hand-on Practice

- Tut3 Hand-on exercise

Discussion

- Discuss how well the standard approach to game playing would apply to games such as tennis, pool, and croquet, which take place in a continuous physical state space.

Assignment 1

- Questions?
- CS188: Adversarial search (Pacman implementation explained)

Question?